



Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУК Информатика и управление

КАФЕДРА ИУКЗ Системы автоматического управления

ОТЧЕТ
по НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ
НА ТЕМУ:

Исследование метода автоматического построения
математической модели системы автоматического
управления по иерархической структурной схеме

Студент группы ИУКЗ-71Б

Исаев А.А.

(Подпись)

(И.О. Фамилия)

Руководитель НИР

Макаренков А.М.

(Подпись)

(И.О. Фамилия)

Оценка руководителя НИР

Баллов, 30-50

(Дата)

Оценка за защиту

Баллов, 30-50

(Дата)

Итоговая оценка

Баллов

(по пятибалльной шкале)

Комиссия: _____

Калуга, 2026

**Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного
учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)**

УТВЕРЖДАЮ
Заведующий кафедрой ИУКЗ
_____ (Д.В. Мельников)
«___» _____ 2026 г.

З А Д А Н И Е

**на научно-исследовательскую работу
на тему:**

Исследование метода автоматического построения математической модели
системы автоматического управления по иерархической структурной схеме

Задание:

1. Проанализировать способы перехода от структурной схемы к математической модели и существующие средства моделирования систем автоматического управления.
2. Формализовать структурную схему как граф и описать метод построения модели в пространстве состояний через матрицу соединений.
3. Описать раскрытие иерархических схем и решение алгебраических контуров с оценкой их обусловленности.
4. Реализовать метод в вычислительном ядре с консольными сценариями исследования.
5. Провести вычислительные эксперименты: корректность на эталонных схемах, алгебраические контуры, иерархия, затраты времени, интегрирование собранной модели; обобщить результаты.

Дата выдачи задания «___» _____ 2026 г.

Руководитель

Макаренков А.М.

(подпись, дата)

(И.О. Фамилия)

Студент

Исаев А.А.

(подпись, дата)

(И.О. Фамилия)

Содержание

Введение.....	4
1. Сбор и анализ материала, анализ литературы.....	6
1.1. Актуальность темы исследования.....	6
1.2. Способы перехода от схемы к модели.....	6
1.3. Объект, предмет, цель и задачи исследования.....	7
2. Проведение научного исследования, расчётов.....	8
2.1. Модели типовых звеньев.....	8
2.2. Схема как граф и матрица соединений.....	8
2.3. Решение по компонентам связности.....	9
2.4. Алгебраические контуры и их обусловленность.....	10
2.5. Иерархические схемы.....	10
2.6. Реализация в вычислительном ядре.....	12
2.7. Эксперимент 1: корректность на эталонных схемах.....	12
2.8. Эксперимент 2: алгебраические контуры.....	13
2.9. Эксперимент 3: иерархические схемы.....	14
2.10. Эксперимент 4: время сборки модели.....	15
2.11. Эксперимент 5: интегрирование собранной модели.....	16
3. Обобщение полученных результатов.....	18
3.1. Научные и практические результаты.....	18
3.2. Ограничения и направления дальнейшей работы.....	18
Заключение.....	19
Список использованных источников.....	20
Приложение А. Листинги программы.....	21

Введение

Научно-исследовательская работа выполнялась по направлению подготовки 27.03.04 «Управление в технических системах» и является исследовательской частью выпускной квалификационной работы «Веб-ориентированная система визуального моделирования систем автоматического управления».

Инженер описывает систему управления структурной схемой: блоки-звенья и стрелки-сигналы, обратные связи, подсистемы внутри подсистем. Чтобы рассчитать такую систему, схему нужно превратить в одну систему уравнений. Вручную это делают правилами структурных преобразований: последовательные звенья перемножают, параллельные складывают, контур с обратной связью сворачивают по формуле $W/(1 + W)$. Для схемы с несколькими пересекающимися контурами и вложенными подсистемами ручное преобразование долгое и легко приводит к ошибке, а программа моделирования должна выполнять его автоматически и без ошибок.

В работе исследуется метод, при котором программа строит модель всей схемы в пространстве состояний $dx/dt = A \cdot x + B \cdot u$, $y = C \cdot x + D \cdot u$ напрямую по графу соединений, без поиска «правильного порядка» сворачивания контуров. Метод реализован в вычислительном ядре разрабатываемой системы.

Цель работы — исследовать метод автоматического построения математической модели САУ по иерархической структурной схеме и проверить его корректность, устойчивость к вычислительным погрешностям и затраты времени.

Для достижения цели решались задачи:

- проанализировать способы перехода от структурной схемы к математической модели и средства моделирования, которые их используют;
- формализовать структурную схему как граф и описать сборку модели через матрицу соединений;

- описать раскрытие иерархических схем (подсистем) и решение алгебраических контуров;
- реализовать метод в вычислительном ядре с консольными сценариями исследования;
- проверить метод на эталонных схемах, исследовать обусловленность алгебраических контуров, работу с иерархией и время сборки;
- сравнить численные методы интегрирования собранной модели.

1. Сбор и анализ материала, анализ литературы

1.1. Актуальность темы исследования

Структурная схема — основной язык теории автоматического управления [2, 3]. Студенту и инженеру удобно думать звеньями, а компьютеру удобно решать систему дифференциальных уравнений. Перевод первого во второе — центральная задача любой программы блочного моделирования.

Наиболее известный инструмент такого моделирования, MATLAB/Simulink, сейчас недоступен российским вузам. Отечественные SimInTech и Engee — коммерческие продукты с закрытым кодом: пользователь не видит, какую модель программа построила по его схеме. Для учебного процесса важна открытая среда, которая показывает собранные матрицы и объясняет, почему схема не может быть рассчитана. Поэтому исследование метода построения модели и его надёжности является актуальной задачей.

1.2. Способы перехода от схемы к модели

Выделены три способа, их сравнение приведено в таблице 1.

Таблица 1 – Способы перехода от структурной схемы к модели

Способ	Как работает	Ограничения
Правила структурных преобразований	Схема сворачивается вручную до одной передаточной функции	Трудоёмко для нескольких контуров; ошибки при ручном расчёте
Поблочное вычисление	На каждом шаге блоки считаются по очереди в порядке прохождения сигнала	Не работает, если сигнал замыкается без инерционного звена (алгебраический контур); нет общих матриц A , B , C , D
Сборка через матрицу соединений (исследуемый метод)	Модели всех звеньев объединяются, связи записываются матрицей, система решается целиком	Только линейные звенья

Поблочный способ был использован в первой версии вычислительного ядра (прототип, весна 2026 г.). Он прост, но отказывается рассчитывать схемы, в которых сигнал проходит по контуру мгновенно, и не даёт матриц модели, по которым определяют устойчивость. Это и стало причиной перехода к исследуемому методу.

1.3. Объект, предмет, цель и задачи исследования

Объект исследования — математические модели линейных непрерывных систем автоматического управления, заданных иерархическими структурными схемами.

Предмет исследования — метод автоматического построения модели в пространстве состояний по графу соединений схемы: его корректность, поведение при алгебраических контурах, работа с вложенными подсистемами и вычислительные затраты.

Цель и задачи исследования сформулированы во введении.

2. Проведение научного исследования, расчётов

2.1. Модели типовых звеньев

Каждое звено описывается своей маленькой моделью в пространстве состояний:

$$dx_i/dt = A_i \cdot x_i + B_i \cdot u_i, \quad y_i = C_i \cdot x_i + D_i \cdot u_i, \quad (1)$$

где u_i — входы звена, y_i — выход, x_i — внутренние переменные (состояния). Например, у усилителя состояний нет, и $y = k \cdot u$ ($D_i = k$), у звена $K/(T \cdot s + 1)$ одно состояние: $dx/dt = -x/T + u/T$, $y = K \cdot x$. Поддерживаемые звенья приведены в таблице 2.

Таблица 2 – Типовые звенья и их модели

Звено	Передаточная функция	Состояни й	Мгновенная передача
Усилитель	k	0	да
Сумматор	$\pm u_1 \pm u_2 \pm \dots$	0	да
Интегратор	k / s	1	нет
Звено 1-го порядка	$K / (T \cdot s + 1)$	1	нет
Колебательное звено	$K \cdot \omega_0^2 / (s^2 + 2\zeta \omega_0 \cdot s + \omega_0^2)$	2	нет
Передаточная функция	$N(s) / D(s)$	$\deg D$	если $\deg N = \deg D$
ФНЧ Баттерворта	порядок n , частота среза ω_c	n	нет
ПИД-регулятор	$K_p + K_i/s + K_d \cdot N \cdot s/(s + N)$	2	да

Столбец «Мгновенная передача» показывает, проходит ли сигнал со входа на выход звена без задержки ($D_i \neq 0$). Это важно: если контур состоит только из таких звеньев, он образует алгебраический контур (раздел 2.4).

2.2. Схема как граф и матрица соединений

Схема рассматривается как ориентированный граф: вершины — блоки, рёбра — связи от выходного порта одного блока к входному порту другого. Каждый вход читает ровно один сигнал: выход другого блока или внешний источник r (ступенчатый вход).

Модели всех звеньев записываются «рядом» — получается блочно-диагональная модель со сложными векторами состояний x , входов u_b и

выходов y_b . Связи схемы тоже линейны и записываются матрицами из нулей и единиц:

$$u_b = M \cdot y_b + N \cdot r, \quad (2)$$

где элемент $M[i, j] = 1$, если i -й вход соединён с j -м выходом. Матрица M — это просто таблица «кто к кому подключён». Подставляя связи в уравнения звеньев, получаем систему для выходов блоков:

$$(I - D_d \cdot M) \cdot y_b = C_d \cdot x + D_d \cdot N \cdot r. \quad (3)$$

Если матрица $I - D_d \cdot M$ невырождена, выходы выражаются через состояния и входы: $y_b = Y_x \cdot x + Y_r \cdot r$, и модель всей схемы получается подстановкой:

$$A = A_d + B_d \cdot M \cdot Y_x, \quad B = B_d \cdot (M \cdot Y_r + N), \quad C = S_y \cdot Y_x, \quad D = S_y \cdot Y_r + S_r, \quad (4)$$

где S_y и S_r описывают, какие сигналы видят осциллографы. Важно, что метод не ищет порядок сворачивания контуров: обратные связи любой сложности просто становятся единицами в матрице M .

На рисунке 1 показана простая схема с обратной связью. Её модель, собранная программой, — $dx/dt = -2 \cdot x + 3 \cdot r$, $y = x$, что совпадает с ручным расчётом по формуле замкнутого контура $W = 3/(s - 1 + 3) = 3/(s + 2)$.

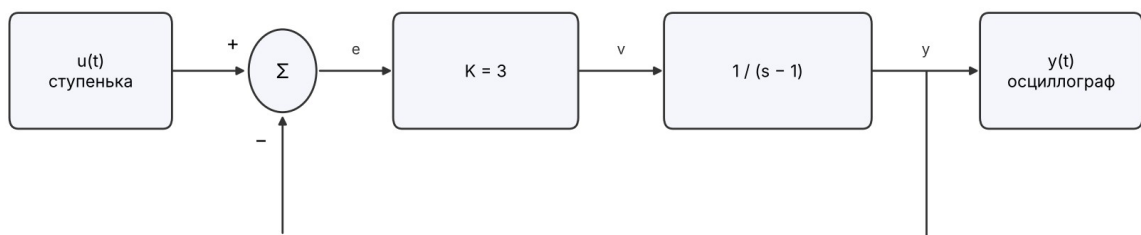


Рисунок 1 – Схема «Стабилизация неустойчивого объекта»

2.3. Решение по компонентам связности

Решать систему $(I - D_d \cdot M) \cdot y_b = \dots$ целиком — неэкономно: в большинстве схем сигнал проходит без контуров, и выходы можно найти простой подстановкой по порядку. Поэтому выходы блоков группируются в

сильно связанные компоненты графа мгновенной передачи $G = D_d \cdot M$ и обрабатываются в топологическом порядке. Компонента из одного блока без петли решается подстановкой, и только настоящий алгебраический контур требует решения системы линейных уравнений (LU-разложение).

2.4. Алгебраические контуры и их обусловленность

Алгебраический контур возникает, когда сигнал обходит контур мгновенно, без инерционного звена. Простейший пример — усилитель с положительной обратной связью: $y = k \cdot (u + y)$, откуда $y = k / (1 - k) \cdot u$. При $k = 1$ решения нет (матрица $I - G$ вырождена), а при k , близком к 1, решение существует, но очень чувствительно: малая погрешность коэффициента k даёт большую погрешность результата.

Чувствительность оценивается числом обусловленности контура:

$$\kappa = (1 + \|G\|) / \sigma_{\min}(I - G), \quad (5)$$

где $\sigma_{\min}$ — наименьшее сингулярное число. Для одной скалярной петли $\kappa = (1 + |k|) / |1 - k|$. Если κ больше $1/\sqrt{\varepsilon} \approx 6,7 \cdot 10^7$ (ε — машинная точность), при решении теряется больше половины значащих цифр. В этом случае программа отказывается от расчёта и называет блоки контура, а не выдаёт недостоверный результат.

2.5. Иерархические схемы

Подсистема — блок, внутри которого находится своя схема с входами «Вход подсистемы» и выходами «Выход подсистемы». Перед сборкой модели схема раскрывается в плоскую: блоки подсистемы копируются на верхний уровень с префиксом пути (например, `measurement::sensor_path::filter`), а связи, проходящие через границу подсистемы, соединяются напрямую. Если выход подсистемы подключён прямо к её входу, связь «проходит насквозь». Два экземпляра одной подсистемы получают разные префиксы и не влияют друг на друга.

Пример многоуровневой схемы показан на рисунках 2 и 3. Модель строится по всей схеме сразу, а в таблице состояний указано, из какой подсистемы пришла каждая переменная (рисунок 4).

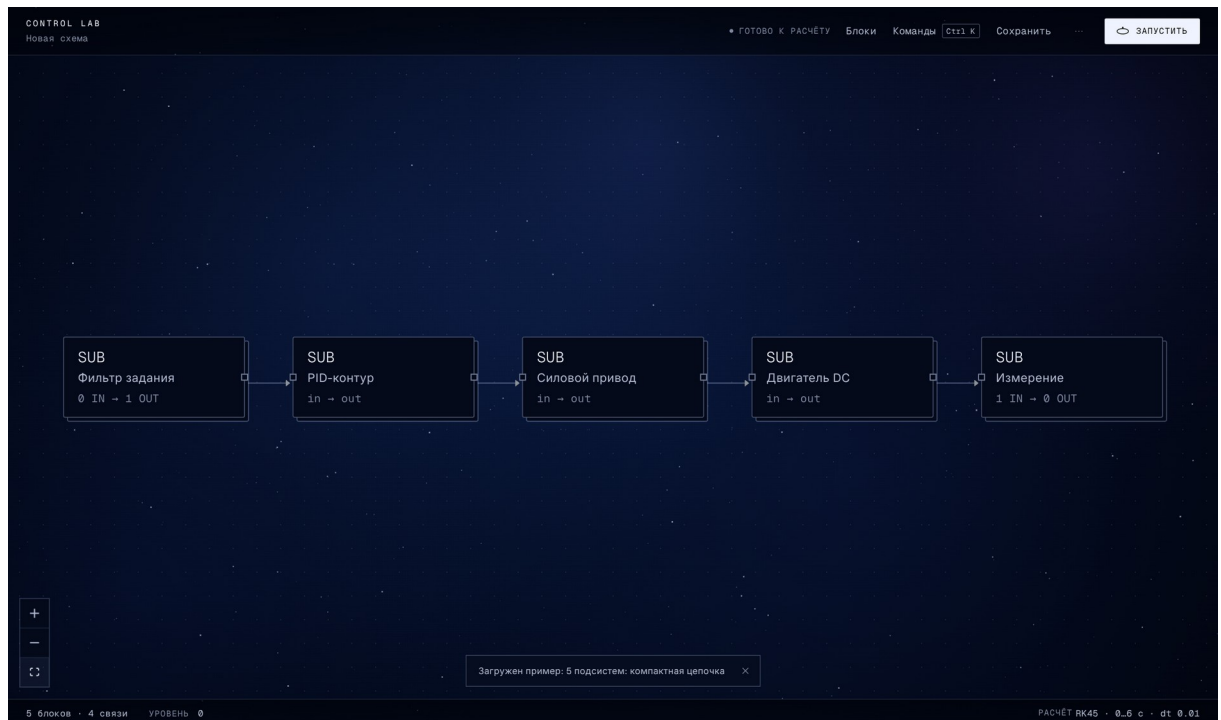


Рисунок 2 – Многоуровневая модель: верхний уровень

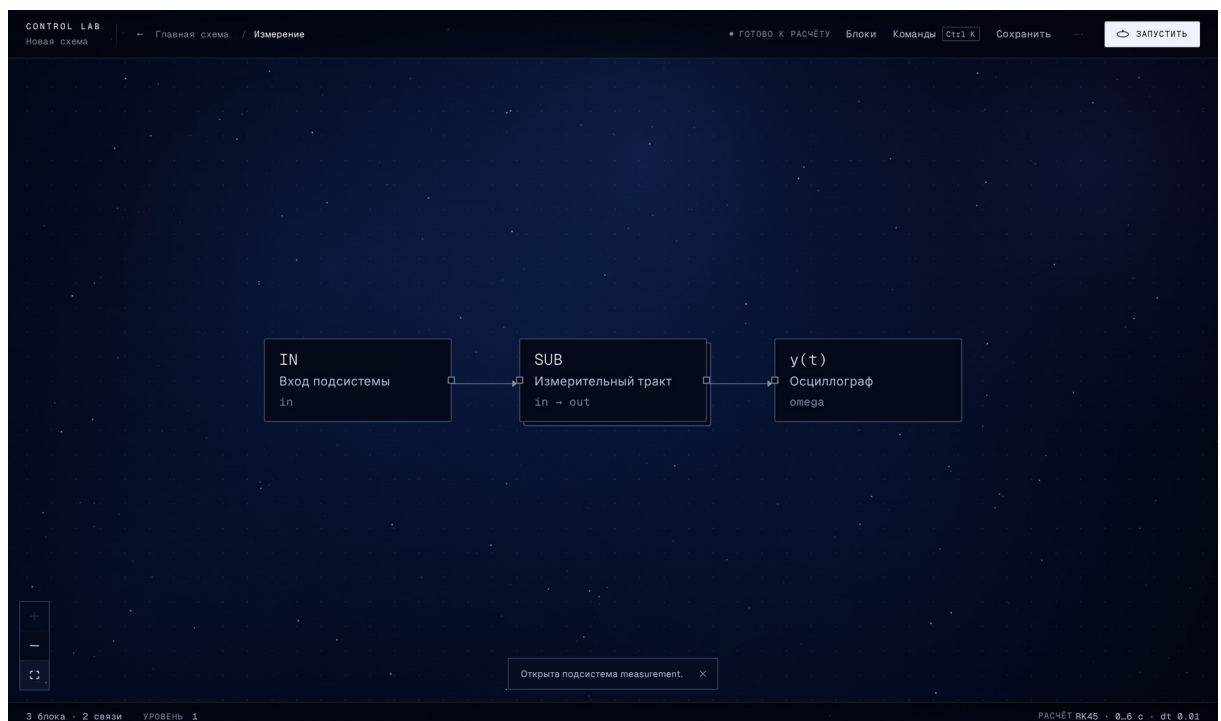


Рисунок 3 – Подсистема «Измерение» с вложенной подсистемой

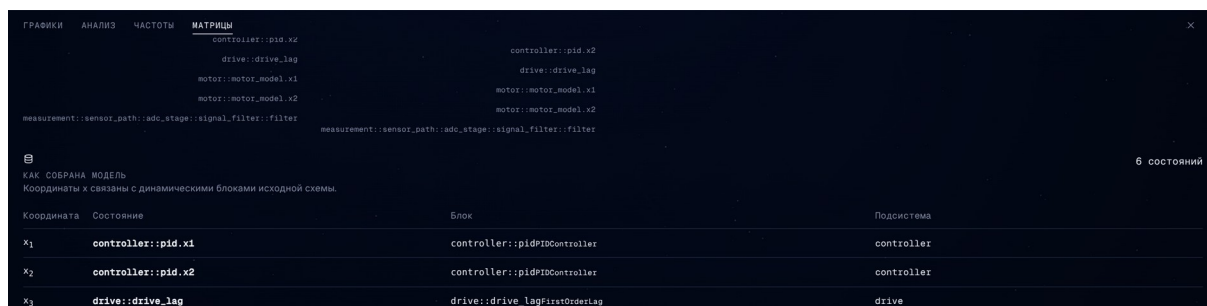


Рисунок 4 – Таблица «Как собрана модель»: состояния и их подсистемы

2.6. Реализация в вычислительном ядре

Метод реализован на Python с использованием NumPy и SciPy [5, 6]. Модули перечислены в таблице 3.

Таблица 3 – Модули вычислительного ядра

Модуль	Назначение
app/core/block_specs.py	описание звеньев, портов и допустимых параметров
app/validation	проверка схемы: типы блоков, порты, один источник на вход
app/simulation/hierarchy.py	раскрытие подсистем в плоскую схему
app/simulation/assembly.py	модели звеньев, матрица соединений, алгебраические контуры, A, B, C, D
app/simulation/solvers.py	численные методы: Эйлер, RK4, RK45, точное решение
scripts/nir_assembly_study.py	консольное исследование метода (раздел 2.7–2.10)
scripts/nir_solver_study.py	консольное исследование численных методов (раздел 2.11)

Исследование запускается одной командой и воспроизводит все таблицы и графики раздела:

```
cd backend
python scripts/nir_assembly_study.py
python scripts/nir_solver_study.py
```

Основная функция сборки приведена в листинге 1 приложения А, решение алгебраических контуров — в листинге 2.

2.7. Эксперимент 1: корректность на эталонных схемах

Для проверки подготовлено 16 эталонных схем: последовательное и параллельное соединение, разветвление, отрицательная и положительная обратная связь, контур с ПИД-регулятором, алгебраические контуры, подсистемы, два экземпляра одной подсистемы, многоканальная подсистема, фильтр Баттерворта. Для каждой схемы передаточная матрица $G(s)$

рассчитана вручную по правилам структурных преобразований. Модель, собранная программой, сравнивалась с ней в четырёх точках s по относительной ошибке $\max|C(sI - A)^{-1}B + D - G(s)| / \max(1, |G(s)|)$.

Результаты приведены в таблице 4. В последнем столбце — результат поблочного подхода первой версии ядра.

Таблица 4 – Сравнение собранных моделей с ручным расчётом

Схема	n	Входы × выходы	Алг. контуры	Ошибка метода	Поблочный подход
series	2	1 × 1	0	1.8e-16	1.8e-16
parallel	2	1 × 1	0	1.6e-16	1.6e-16
branching	1	1 × 2	0	0.0e+00	0.0e+00
negative_feedback	2	1 × 1	0	4.7e-16	4.7e-16
positive_feedback	1	1 × 1	0	0.0e+00	0.0e+00
pid_loop	3	1 × 1	0	6.1e-16	6.1e-16
biproper_loop	1	1 × 1	1	1.1e-16	не собирает
static_loop	0	1 × 1	1	0.0e+00	не собирает
subsystem_feedback	1	1 × 1	0	1.1e-16	1.1e-16
flat_subsystem_feedback	1	1 × 1	0	1.1e-16	1.1e-16
two_instances	2	1 × 1	0	5.6e-17	5.6e-17
two_instances_with_inner_scopes	2	1 × 3	0	5.6e-17	5.6e-17
pass_through	1	1 × 1	0	0.0e+00	0.0e+00
nested_feedback	1	1 × 1	0	1.1e-16	1.1e-16
mimo_subsystem	1	2 × 2	0	0.0e+00	0.0e+00
butterworth	4	1 × 1	0	8.9e-16	8.9e-16

Исследуемый метод собрал все 16 схем, наибольшая ошибка — 8.9e-16, то есть на уровне ошибок округления чисел в компьютере. Поблочный подход не смог собрать 2 схемы — обе содержат алгебраический контур.

2.8. Эксперимент 2: алгебраические контуры

Исследовался контур $y = k \cdot (u + y)$ при приближении коэффициента k к единице. Для каждого k сравнивались решение программы и точное $y = k / (1 - k)$, а также измерялось, во сколько раз относительная погрешность коэффициента k (10^{-9}) усиливается в результате. Результаты приведены в таблице 5 и на рисунке 5.

Таблица 5 – Решение алгебраического контура $y = k \cdot (u + y)$

k	k (петля)	k (программа)	y программы	y точное	Ошибка	Усиление	Итог
---	-----------	---------------	-------------	----------	--------	----------	------

0.5	3	7.12	1	1	0.0e+00	2	решён
0.9	19	39	9	9	0.0e+00	10	решён
0.99	199	399	99	99	0.0e+00	100	решён
0.999	2e+03	4e+03	999	999	0.0e+00	1e+03	решён
0.999999	2e+06	4e+06	999999	999999	0.0e+00	1e+06	решён
0.999999999 999	2e+12	—	—	1e+12	—	—	отказ: плохо обусловлен
1	inf	—	—	inf	—	—	отказ: вырожден
1.5	5	11.4	-3	-3	1.5e-16	2	решён
3	2	6.83	-1.5	-1.5	1.5e-16	0.5	решён

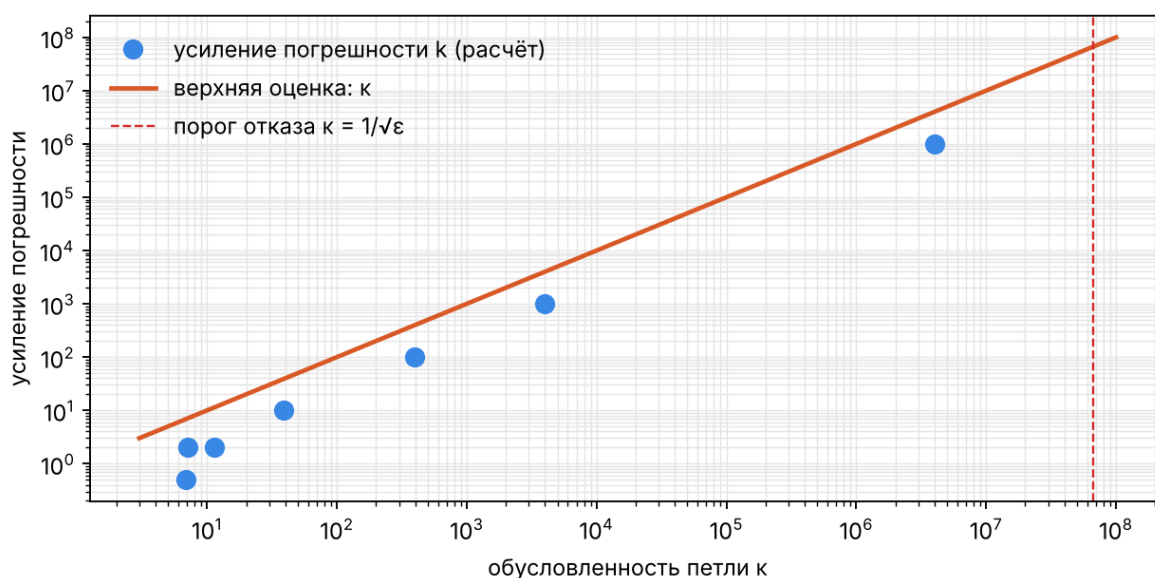


Рисунок 5 – Усиление погрешности в зависимости от обусловленности петли

Пока контур хорошо обусловлен, программа решает его точно. С приближением k к 1 обусловленность растёт как $1/(1 - k)$, и вместе с ней растёт усиление погрешности — точки на рисунке лежат ниже оценки k , то есть оценка надёжна. При $k = 1 - 10^{-12}$ ($k \approx 2 \cdot 10^{12}$) программа отказывается от расчёта, потому что результат был бы недостоверным, а при $k = 1$ сообщает, что контур не имеет решения. Значение k , вычисленное программой, примерно вдвое больше скалярной оценки: в схеме контур проходит через два блока (сумматор и усилитель), и оценка учитывает оба сигнала.

2.9. Эксперимент 3: иерархические схемы

Сначала сравнивались две записи одной системы: контур, проходящий через подсистему, и та же схема без подсистем. Наибольшее расхождение выходов составило 0.0e+00. Затем звено 1-го порядка в контуре с обратной связью вкладывалось в подсистемы глубиной от 1 до 32 уровней (таблица 6).

Таблица 6 – Раскрытие вложенных подсистем

Глубина	Блоков после раскрытия	n	Раскрытие, мс	Сборка, мс	Отличие y(t)
1	4	1	0,08	0,42	0.0e+00
2	4	1	0,10	0,44	0.0e+00
4	4	1	0,13	0,52	0.0e+00
8	4	1	0,16	0,73	0.0e+00
16	4	1	0,28	0,98	0.0e+00
32	4	1	0,62	1,54	0.0e+00

При любой глубине после раскрытия остаются те же четыре блока, модель первого порядка и в точности тот же выход. Время раскрытия растёт линейно с глубиной и даже при 32 уровнях не превышает долей миллисекунды.

2.10. Эксперимент 4: время сборки модели

Время сборки измерялось на цепочке звеньев 1-го порядка длиной от 2 до 256 звеньев — без контуров и с отрицательной обратной связью вокруг каждого звена (медиана 15 запусков). Результаты приведены в таблице 7 и на рисунке 6.

Таблица 7 – Время сборки модели

n	Блоков	Сборка, мс	Блоков (с контурами)	Сборка, мс (с контурами)
2	4	0,31	6	0,41
8	10	0,55	18	1,18
32	34	1,82	66	3,42
64	66	3,05	130	6,84
128	130	6,60	258	17,55
256	258	18,60	514	54,33

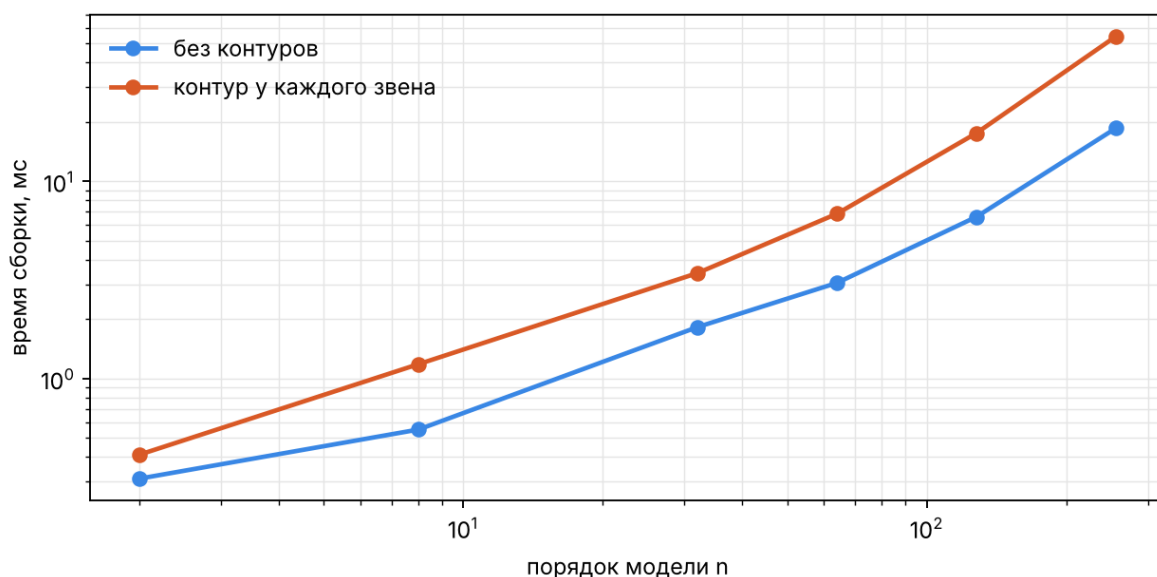


Рисунок 6 – Время сборки модели в зависимости от порядка

Даже модель 256-го порядка из более чем 500 блоков с 256 контурами собирается быстрее чем за десятую долю секунды. Для учебных схем (до десятков блоков) время сборки — доли миллисекунды, что позволяет пересобирать модель при каждом движении ползунка параметра.

2.11. Эксперимент 5: интегрирование собранной модели

Собранная модель $dx/dt = A \cdot x + B \cdot r$ решается численно. Сравнивались метод Эйлера, метод Рунге–Кутты 4-го порядка (RK4) и метод RK45 с автоматическим выбором шага. Эталоном служило точное решение через матричную экспоненту, которое возможно именно потому, что метод даёт матрицы A и B.

Наблюдаемый порядок точности (таблица 8) совпал с теорией: около 1 для метода Эйлера и около 4 для RK4.

Таблица 8 – Ошибка методов на контуре с ПИД-регулятором

h, с	Эйлер	RK4	RK45	p Эйлер	p RK4
0,2	1.20e+02	3.89e-03	7.35e-09	—	—
0,1	2.11e+00	2.37e-04	7.35e-09	5,83	4,04
0,05	2.75e-01	1.46e-05	7.35e-09	2,94	4,02
0,025	1.06e-01	9.08e-07	7.37e-09	1,38	4,01
0,0125	4.74e-02	5.65e-08	7.43e-09	1,15	4,01

Для ошибки не больше 10^{-3} методу Эйлера нужно в 128–512 раз больше шагов, чем RK4, поэтому в системе используются RK4 и RK45, а метод Эйлера оставлен как эталон для исследования.

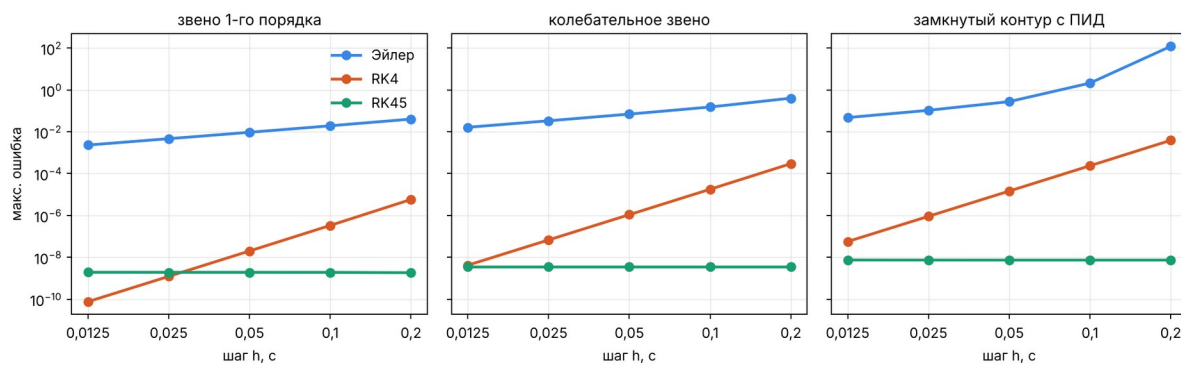


Рисунок 7 – Ошибка численных методов в зависимости от шага

3. Обобщение полученных результатов

3.1. Научные и практические результаты

В ходе НИР получены следующие результаты:

- формализована структурная схема как граф и описан метод сборки модели в пространстве состояний через матрицу соединений, не требующий ручного сворачивания контуров;
- описаны раскрытие иерархических схем и решение алгебраических контуров с оценкой их обусловленности;
- на 16 эталонных схемах ошибка метода не превысила $9e-16$, тогда как поблочный подход не справился с 2 схемами;
- показано, что оценка обусловленности надёжно ограничивает усиление погрешности, и обоснован порог отказа $\kappa > 1/\sqrt{\epsilon}$;
- подтверждено, что вложенность подсистем не меняет модель, а время раскрытия растёт линейно;
- модель 256-го порядка собирается за десятки миллисекунд;
- экспериментально подтверждены порядки точности численных методов.

Практический результат — вычислительное ядро веб-системы визуального моделирования, которое строит модель любой линейной схемы автоматически и показывает пользователю матрицы A, B, C, D и их связь с блоками схемы.

3.2. Ограничения и направления дальнейшей работы

- метод рассчитан на линейные звенья; нелинейные элементы (насыщение, реле) потребуют другой формы модели;
- входные воздействия ограничены ступенчатыми;
- на основе собранной модели в рамках курсового проекта разрабатывается модуль частотного анализа устойчивости (размыкание контура, критерий Найквиста, запасы устойчивости).

Заключение

В результате НИР исследован метод автоматического построения математической модели системы автоматического управления по иерархической структурной схеме.

Метод представляет схему графом, связи — матрицей соединений и получает модель всей схемы в пространстве состояний решением системы линейных уравнений по компонентам связности. Подсистемы раскрываются в плоскую схему, алгебраические контуры решаются с проверкой обусловленности.

Вычислительные эксперименты показали, что метод совпадает с ручным расчётом по правилам структурных преобразований на уровне ошибок округления, собирает схемы, недоступные поблочному подходу, корректно отказывается от плохо обусловленных контуров, не зависит от глубины вложенности подсистем и работает быстро даже для моделей сотен порядков.

Метод реализован в вычислительном ядре веб-системы визуального моделирования и является основой выпускной квалификационной работы.

Список использованных источников

1. Финошин А.В., Корнюшин Ю.П., Краснощеченко В.И. Научно-исследовательская работа: практикум. – Калуга: КФ МГТУ им. Н.Э. Баумана, 2024. – 17 с.
2. Бесекерский В.А., Попов Е.П. Теория систем автоматического регулирования. – М.: Директмедиа, 2021. – 747 с.
3. Ким Д.П. Теория автоматического управления. Т. 1. Линейные системы. – М.: Физматлит, 2016. – 312 с.
4. Голуб Дж., Ван Лоун Ч. Матричные вычисления. – М.: Мир, 1999. – 548 с.
5. SciPy API Reference [Электронный ресурс]. – URL: <https://docs.scipy.org/doc/scipy/reference/> (дата обращения: 01.10.2026).
6. NumPy Documentation [Электронный ресурс]. – URL: <https://numpy.org/doc/> (дата обращения: 01.10.2026).
7. Hairer E., Nørsett S.P., Wanner G. Solving Ordinary Differential Equations I: Nonstiff Problems. – Berlin; Heidelberg: Springer, 2008. – 528 p.
8. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. 3-е изд. – М.: Вильямс, 2013. – 1328 с.

Приложение А. Листинги программы

Листинг 1 – Сборка модели схемы (app/simulation/assembly.py)

```
def assemble_linear_model(flat: Diagram) -> LinearModel:
    """Assemble a validated, flattened diagram into (A, B, C, D)."""

    sources: list[StepSource] = []
    scopes: list[ScopeOutput] = []
    source_index: dict[str, int] = {}
    scope_index: dict[str, int] = {}

    dynamic: list[tuple[Block, BlockRealization]] = []
    output_index: dict[str, int] = {} # block id -> row of y_b
    input_offset: dict[str, int] = {} # block id -> first row of u_b
    states: list[StateEntry] = []
    total_inputs = 0

    for block in flat.blocks:
        if block.type in SOURCE_TYPES:
            source_index[block.id] = len(sources)
            sources.append(
                StepSource(
                    block.id,
                    get_numeric_parameter(block.parameters, "amplitude", 1.0),
                    get_numeric_parameter(block.parameters, "t0", 0.0),
                )
            )
            continue
        if block.type in SINK_TYPES:
            label = str(block.parameters.get("label") or "").strip() or block.id
            reference = (
                get_numeric_parameter(block.parameters, "reference", 0.0)
                if "reference" in block.parameters
                else None
            )
            scope_index[block.id] = len(scopes)
            scopes.append(ScopeOutput(block.id, label, reference))
            continue

        realization = block_realization(block)
        if realization.b.shape[1] != len(block.input_ports):
            raise ModelAssemblyError(
                [f"Блок '{block.id}': число входов не совпадает с моделью звена."]
            )
        output_index[block.id] = len(dynamic)
        input_offset[block.id] = total_inputs
        total_inputs += len(block.input_ports)
        for local in range(realization.order):
            label = block.id if realization.order == 1 else f"{block.id}.x{local + 1}"
            states.append(StateEntry(block.id, block.type, local, label))
        dynamic.append((block, realization))

    realizations = [realization for _, realization in dynamic]
    n = sum(r.order for r in realizations)
    n_out = len(dynamic)
    n_src = len(sources)
    p = len(scopes)

    a_d = _block_diagonal([r.a for r in realizations], n, n)
    b_d = _block_diagonal([r.b for r in realizations], n, total_inputs)
    c_d = _block_diagonal([r.c for r in realizations], n_out, n)
    d_d = _block_diagonal([r.d for r in realizations], n_out, total_inputs)
    x0 = np.concatenate([r.x0 for r in realizations]) if realizations else np.zeros(0)

    m = np.zeros((total_inputs, n_out))
    nr = np.zeros((total_inputs, n_src))
    s_y = np.zeros((p, n_out))
    s_r = np.zeros((p, n_src))
```

```

blocks_by_id = {block.id: block for block in flat.blocks}

for connection in flat.connections:
    target = blocks_by_id[connection.to_block]
    if target.type in SINK_TYPES:
        row_matrix_y, row_matrix_r, row = s_y, s_r, scope_index[target.id]
    else:
        row = input_offset[target.id] +
target.input_ports.index(connection.to_port)
        row_matrix_y, row_matrix_r = m, nr
        if connection.from_block in source_index:
            row_matrix_r[row, source_index[connection.from_block]] = 1.0
        else:
            row_matrix_y[row, output_index[connection.from_block]] = 1.0

g = d_d @ m
owners = [block.id for block, _ in dynamic]
y_solution, loops = _solve_algebraic_equations(g, np.hstack([c_d, d_d @ nr]),
owners)
y_x, y_r = y_solution[:, :n], y_solution[:, n:]

return LinearModel(
    a=a_d + b_d @ m @ y_x,
    b=b_d @ (m @ y_r + nr),
    c=s_y @ y_x,
    d=s_y @ y_r + s_r,
    x0=x0,
    states=states,
    sources=sources,
    scopes=scopes,
    algebraic_loops=loops,
)

```

Листинг 2 – Решение алгебраических контуров (app/simulation/assembly.py)

```

def _solve_algebraic_equations(
    g: np.ndarray,
    rhs: np.ndarray,
    owners: list[str],
) -> tuple[np.ndarray, list[AlgebraicLoop]]:
    """Solve (I - g) Y = rhs component by component."""

    solution = np.zeros_like(rhs)
    loops: list[AlgebraicLoop] = []
    eps = np.finfo(float).eps
    for component in _topological_components(g):
        index = np.asarray(component)
        local_rhs = rhs[index] + g[index] @ solution # unsolved rows of solution are
still zero
        if index.size == 1 and g[index[0], index[0]] == 0.0:
            solution[index] = local_rhs
            continue

        loop_blocks = sorted({owners[i] for i in component})
        loop_gain = g[np.ix_(index, index)]
        matrix = np.eye(index.size) - loop_gain
        smallest = float(np.linalg.svd(matrix, compute_uv=False)[-1])
        # Relative sensitivity of the loop solution to perturbations of the
        # loop coefficients: kappa = (1 + ||G||) / sigma_min(I - G). For a
        # scalar loop y = k y + ... this is (1 + |k|) / |1 - k|.
        scale = 1.0 + float(np.linalg.norm(loop_gain, 2))
        if smallest <= eps * scale * index.size:
            raise ModelAssemblyError(
                [
                    "Алгебраическая петля не имеет решения (матрица I - D·M
вырождена): "
                    + " → ".join(loop_blocks)
                    + ". Измените коэффициенты петли или добавьте динамическое звено."
                ]
            )
    return solution, loops

```

```

    )
    condition = scale / smallest
    if condition > MAX_LOOP_CONDITION:
        raise ModelAssemblyError(
            [
                f"Алгебраическая петля плохо обусловлена (κ = {condition:.3g}): "
                + " → ".join(loop_blocks)
                + ". Результат расчёта был бы недостоверным."
            ]
        )
    solution[index] = lu_solve(lu_factor(matrix), local_rhs)
    loops.append(AlgebraicLoop(blocks=loop_blocks, condition_number=condition))
    return solution, loops

```

Листинг 3 – Эксперимент с алгебраическим контуром

(scripts/nir_assembly_study.py)

```

def algebraic_loops() -> tuple[list[dict], list[tuple[float, float, float]]]:
    """Положительная обратная связь без динамики:  $y = k \cdot (u + y)$ , точное решение  $y = k/(1 - k)$ ."""
    rows, curve = [], []
    for k in (0.5, 0.9, 0.99, 0.999, 0.999999, 1.0 - 1e-12, 1.0, 1.5, 3.0):
        case = static_loop_case(k, ("+", "+"))
        exact = k / (1.0 - k) if k != 1.0 else float("inf")
        formula = (1.0 + abs(k)) / abs(1.0 - k) if k != 1.0 else float("inf")
        row = {"k": f"{k:.12g}", "κ скалярной петли": f"{formula:.3g}"}
        try:
            model = compile_model(Diagram.model_validate(case.diagram)).model
            value = float(model.d[0, 0])
            kappa = model.algebraic_loops[0].condition_number
            error = abs(value - exact) / max(1.0, abs(exact))
            # Чувствительность: насколько относительная погрешность κ усиливается в
            # решении петли.
            delta = 1e-9
            shifted = compile_model(Diagram.model_validate(static_loop_case(k * (1 +
            delta), ("+", "+")).diagram)).model
            amplification = abs(float(shifted.d[0, 0]) - value) / abs(value) / delta
            row.update({"κ (программа)": f"{kappa:.3g}", "у программы":
            f"{value:.10g}",
            "у точное": f"{exact:.10g}", "отн. ошибка": f"{error:.1e}",
            "усиление погрешности κ": f"{amplification:.3g}", "итог":
            "решён"})
            curve.append((kappa, amplification, k))
        except DiagramCompilationError as exc:
            reason = "вырожден" if "не имеет решения" in exc.errors[0] else "плохо
            обусловлен"
            row.update({"κ (программа)": "-", "у программы": "-", "у точное":
            f"{exact:.4g}",
            "отн. ошибка": "-", "усиление погрешности κ": "-", "итог":
            f"отказ: {reason}"})
            rows.append(row)
    return rows, curve

```

Листинг 4 – Метод Эйлера (app/simulation/solvers.py)

```

def euler_integrate(
    rhs: Rhs,
    x0: np.ndarray,
    t_grid: np.ndarray,
    *,
    breakpoints: Sequence[float] = (),
) -> np.ndarray:
    """Explicit Euler on the given grid:  $x_{k+1} = x_k + h \cdot f(t_k, x_k)$ .

    First-order reference method for the numerical study; it is not offered by

```

```

the API. The derivative is taken at the left end of each step, so a jump at
a breakpoint grid node takes effect from the step that starts there.
"""

_validate_time_grid(t_grid)
del breakpoints # the left-point rule never evaluates past the step start
if x0.size == 0:
    return np.zeros((0, t_grid.size))

trajectory = np.zeros((x0.size, t_grid.size))
trajectory[:, 0] = x0
x = x0.astype(float).copy()
for index in range(t_grid.size - 1):
    left, right = float(t_grid[index]), float(t_grid[index + 1])
    with np.errstate(over="ignore", invalid="ignore"): # divergence is reported
        below
            x = x + (right - left) * rhs(left, x)
            if not np.all(np.isfinite(x)):
                raise SolverError(f"Эйлер: состояние перестало быть конечным в момент
t={right:g}.")
            trajectory[:, index + 1] = x
    return trajectory

```